



Business-Adaptive Voice AI

Conversation-Driven LLM Personalization and Style-Aware Speech Synthesis for AI Phone Agents

A proposal to extend an existing open-source AI phone-calling platform with per-business LLM fine-tuning (LoRA/QLoRA), hybrid retrieval-augmented generation, and a personalized Voice Style Profile for text-to-speech delivery.

Base project	Open-Source AI Calling Platform
Proposal type	Final Year Project (FYP) Proposal
Focus areas	LLM Personalization & Fine-Tuning, RAG, Voice AI / TTS
Date	September 2026

Table of Contents

1. The Current Project	3
2. Current Architecture	3
3. Problems and Limitations	4
4. Proposed FYP System	5
5. Proposed Architecture	5
6. LLM Fine-Tuning / Personalization Approach	6
7. TTS / Voice Personalization & the Voice Style Profile	8
8. Differentiation from a Normal Chatbot / RAG System	9
9. Research Novelty vs. Engineering	10
10. Research Questions	10
11. Evaluation Methodology	11
12. Expected Results	11
13. FYP Scope and Feasibility	12
14. Implementation Roadmap	12
15. Risks and Mitigations	13
16. Conclusion & References	13

Abstract

The base project is a working, open-source AI phone-calling platform: a caller rings a real number, an LLM-backed agent transcribes, reasons, and replies over a live SIP/WebRTC call, with retrieval-based knowledge lookup, function-calling tools, and a bilingual (English/Arabic) speech stack. Its core limitation is that every business's agent is the *same* shared foundation model, differentiated only by a hand-written system-prompt string, and speaks with a hand-picked stock voice with no control over delivery. This proposal extends it into a **business-adaptive voice AI platform**: businesses fine-tune a lightweight adapter on their own historical call transcripts so the agent learns their terminology, policies, and communication style, combined with the platform's existing retrieval system for facts that change too often to bake into weights; and businesses receive a **Voice Style Profile** that lets their agent's delivery — pace, pauses, emphasis, tone — stay consistent with their brand. The contribution is not any single technique (LoRA, RAG, neural TTS are all established) but

the **evaluated combination and its data pipeline** — turning raw call logs into training data and a style profile, and measuring, honestly, when personalization is worth it against retrieval alone.

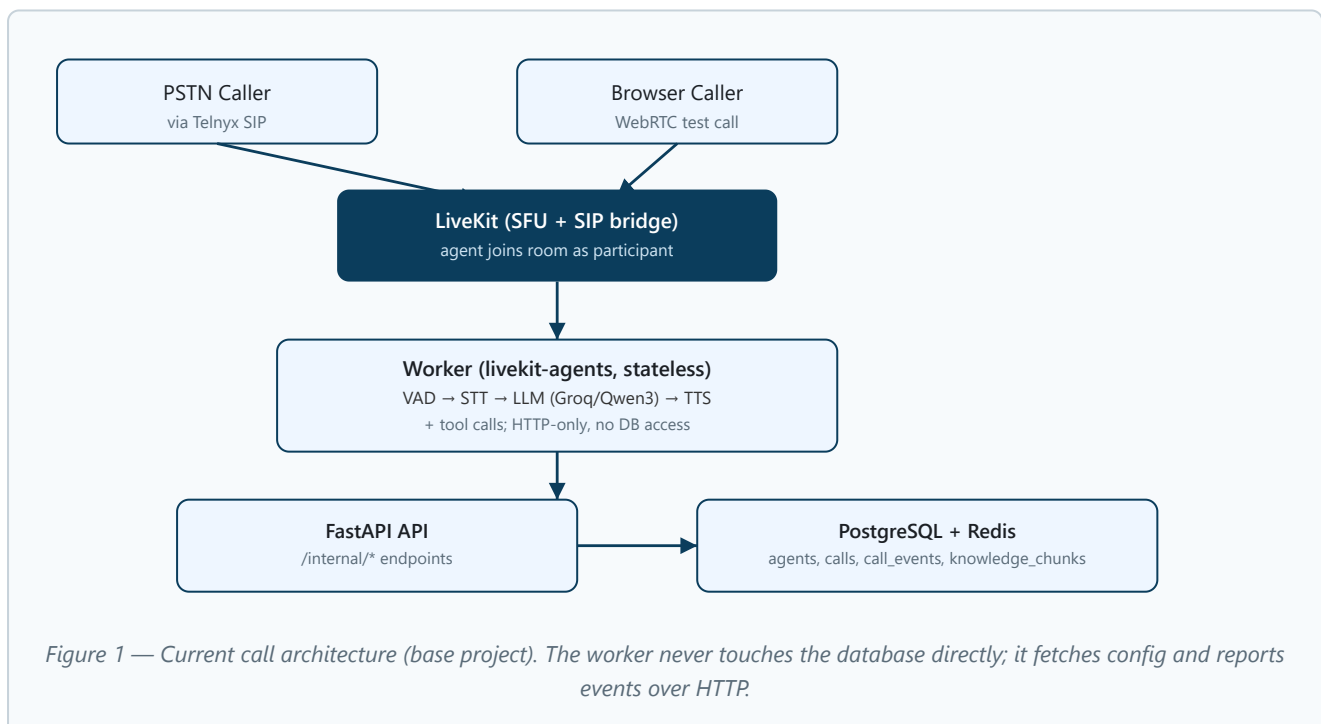
1. The Current Project

The base project is a multi-tenant, self-hostable platform for building and operating AI voice agents on real phone lines. An organization creates an "agent" (system prompt, voice, tools, knowledge access), attaches a phone number, and the agent holds live conversations with callers. It is an implemented, working system — not a prototype — built on FastAPI + PostgreSQL, a stateless `livekit-agents` Python worker running the voice pipeline, and a Next.js dashboard.

1.1 What already works (verified against source code)

- Real inbound/outbound PSTN calls via LiveKit + Telnyx SIP, plus free browser test calls.
- STT (Groq-hosted Whisper, or LiveKit's streaming gateway), LLM (Groq-hosted Qwen3, model/temperature configurable per agent), TTS (Groq Orpheus, Fish Audio streaming, or self-hosted Chatterbox).
- Function-calling tools: end call, transfer call, send DTMF, and a generic outbound-webhook tool for CRM/booking integrations.
- A working retrieval pipeline: uploaded documents are chunked, embedded with a multilingual model, and searched by cosine similarity when the LLM calls a `lookup_knowledge` tool mid-conversation.
- Full call transcripts, recordings, post-call LLM analysis, and cost/latency analytics per call.
- Native bilingual (English/Arabic) design, including RTL-aware UI and Arabic-specific STT/TTS models.

2. Current Architecture



Knowledge retrieval is a plain cosine-similarity search over embedded document chunks, invoked as an LLM tool call mid-conversation, not injected into the system prompt at session start. Every organization's

agent runs the same shared Groq-hosted LLM, differentiated only by its system-prompt text; TTS is invoked with plain text and a fixed voice identifier. Full detail is provided in the accompanying [docs/architecture.md](#).

3. Problems and Limitations in the Current System

#	Limitation	Why it matters
1	No LLM personalization. Every org uses the identical shared Groq model; the only per-business signal is a hand-written prompt string.	The agent cannot learn a business's actual phrasing, policies, or common-intent handling from calls it has already had — that data is stored and never used.
2	No TTS delivery control. TTS calls carry text and a fixed voice ID only.	Every business sounds like a stock voice reading text — no way to express brand personality or match delivery to context.
3	Knowledge base is basic and org-wide. Fixed-size chunking, brute-force cosine search, no per-agent scoping, no vector index.	Adequate today; a real ceiling as knowledge bases grow, and does nothing for delivery style.
4	No feedback loop from real calls back into agent quality.	Transcripts and analysis are captured but never used to improve the model or flag style drift.
5	Platform reliability/observability gaps (call-state bugs, unfinished auth lifecycle, no logging) — documented in the independent engineering audit.	Out of scope for this proposal; noted so scope decisions can deliberately avoid depending on unfinished layers.

Limitations 1–2 are the direct target of this proposal; 3 is extended, not rebuilt; 4 is the resource being unlocked; 5 is explicitly out of scope.

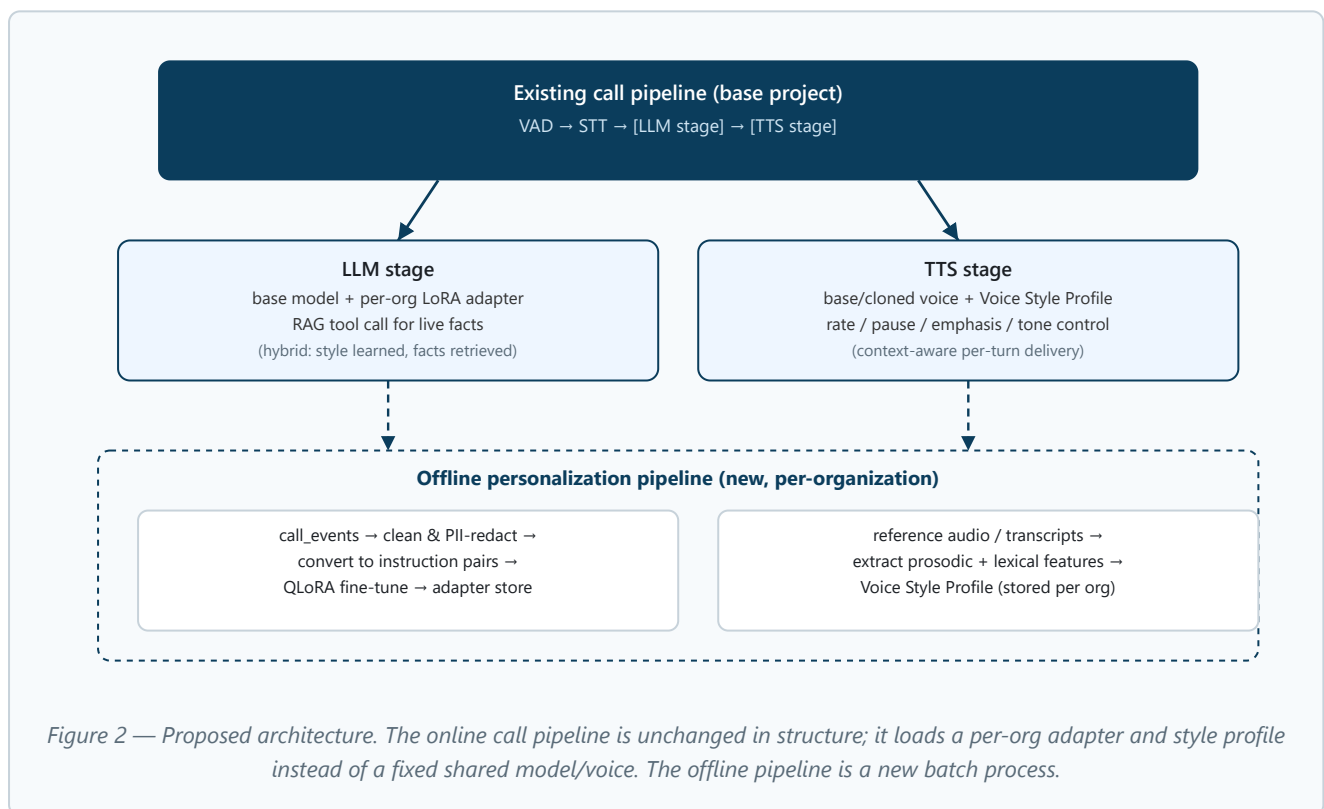
4. Proposed FYP System

Business-Adaptive Voice AI: businesses upload or accumulate historical call transcripts and business documents, and the platform learns two separate things from that data:

- **What the business knows and how it responds** — via a lightweight fine-tuned adapter (LoRA/QLoRA) on an open-weight LLM, trained on that business's cleaned conversation history, combined with the existing retrieval system for fast-changing facts.
- **How the business sounds** — via a **Voice Style Profile**: a structured, per-business record of speaking rate, pause behavior, emphasis patterns, and tone-per-intent, applied automatically at synthesis time on a selected or cloned base voice.

The rest of the platform (telephony, call handling, dashboard, tools) is unchanged — this extends the LLM and TTS stages of the existing pipeline, it does not replace the product.

5. Proposed Architecture



6. LLM Fine-Tuning / Personalization Approach

6.1 Training data: from raw transcripts to training pairs

1. **Source:** `call_events` rows (user/agent turns), grouped by call, ordered by timestamp — this data already exists per organization.
2. **Filtering:** drop failed/very short calls and calls flagged low-quality by existing post-call analysis; deduplicate boilerplate greetings/closings.
3. **PII redaction:** regex/NER pass over phone numbers, names, addresses, and payment-related utterances before any data leaves the raw-transcript store.
4. **Conversion:** each turn becomes an instruction-tuning example — `{system_prompt, conversation history, target_reply}` in a standard chat-template JSONL format, tagged with an intent label for stratified sampling and evaluation.
5. **Style feature extraction:** per-business lexical statistics (reply length, formality markers, characteristic phrases, greeting/closing patterns) — doubling as conditioning input for fine-tuning and as the seed for the Voice Style Profile (§7), tying the "what" and "how" pipelines to one data source.

6.2 Data volume — an honest constraint

A single SMB's real call history is unlikely to provide the thousands of clean turns a fine-tune ideally wants. This is treated as a scoping fact: the project evaluates on (a) a small real/pilot dataset from test agents built during the project, and (b) a validated synthetic-augmentation set (LLM-generated, style-consistency filtered), explicitly reported as a limitation on real-world generalizability.

6.3 Model and training method

- **Base model:** a small open instruction-tuned model realistic for single-GPU/Colab-class compute — e.g., Qwen2.5-7B-Instruct or Llama-3.1-8B-Instruct — deliberately not the 32B model the base platform runs live.
- **Method:** QLoRA (4-bit NF4 base + LoRA adapters on attention/MLP projections, via PEFT + bitsandbytes + a standard SFT trainer).
- **Per-organization adapters:** one small adapter per business (tens of MB) on a shared quantized base.
- **Serving for the prototype:** adapters evaluated via a local inference server, since the base platform's live LLM stage is a hosted Groq endpoint that does not support custom weights; production dynamic multi-LoRA serving (e.g., vLLM/SGLang) is discussed as future work, not built.

6.4 RAG vs. Fine-Tuning vs. Hybrid

Dimension	RAG only (current)	Fine-tuning only	Hybrid (proposed)
Fact freshness	High	Low — frozen at training time	High — facts still retrieved live
Hallucination on facts	Low	Higher — can confidently state stale/invented facts	Low
Style/tone control	Weak — inconsistent under real pressure	Strong — learned into weights	Strong
Implicit recurring intents/policies	Weak — only what's literally documented	Strong	Strong
Data/maintenance cost	Low	Higher — retrain as behavior evolves	Medium
Realistic at SMB data volume	Yes	Marginal	Yes — FT workload is style-only, smaller

Recommendation: hybrid, not fine-tuning-first. Fine-tuning suits behavior that is stable and recurring — phrasing, tool-use habits, intent handling — because the pattern repeats across training examples and generalizes. It is poorly suited to facts that change, since a model cannot "forget" a stale trained fact and retraining on every data change is not viable. RAG has the opposite profile. The hybrid design keeps the base platform's existing retrieval responsible for facts and adds fine-tuning responsible for style and behavior — each technique used for what it is actually good at. Fine-tuning is *not* assumed superior; §11 defines the experiment that tests this claim.

6.5 Evaluation of the LLM component

- **Automatic:** held-out perplexity/next-turn accuracy, tool-call format accuracy, guardrail adherence, hallucination rate on a factual-QA set with known answers.
- **Style fidelity:** embedding-based stylistic distance to real historical replies, lexical-overlap on characteristic phrases.
- **Human/LLM-judge preference:** pairwise comparison (RAG-only / fine-tune-only / hybrid) on style match, factual correctness, naturalness, using a fixed rubric.

7. TTS / Voice Personalization — the Voice Style Profile

This area is given deliberate special weight: it is the platform's least-developed dimension today (TTS is currently "text + fixed voice ID, nothing else") and the least explored area among comparable open-source voice-agent projects.

7.1 What a Voice Style Profile is

Attribute	Source	Effect at synthesis time
Base voice	Selected stock voice, or cloned from a short reference clip	Which voice model/embedding is used
Speaking rate	Extracted from reference audio (words/min) or set manually	Rate parameter passed to the TTS engine
Pause behavior	Extracted pause statistics after greetings, before key info	Pause markers in delivery-annotated text
Emphasis	Business/product names, key terms	Emphasis markup around those tokens
Pronunciation overrides	Business/product name pronunciations (TTS-side analogue of the existing STT <code>boosted_keywords</code> field)	Phoneme/lexicon override at synthesis
Tone-by-intent	Default tone mapped to coarse intent (apology, sales, confirmation)	Emotion/intensity setting selected per utterance, not globally fixed

7.2 How it is built and applied

1. **Extraction:** from a small set of reference audio (a business's IVR recordings, or a short scripted reading) using standard prosodic-analysis tooling (pitch, rate, pause extraction) to seed profile defaults; manual override UI as fallback.
2. **Context-aware delivery at call time:** a lightweight **delivery controller** tags the LLM's outgoing text with the profile's settings plus a coarse per-turn intent/emotion signal, producing delivery-annotated text (pause/emphasis/rate markers) before synthesis.
3. **Synthesis substrate:** implemented against the base platform's existing self-hosted Chatterbox integration rather than a new engine — Chatterbox already supports zero-shot voice cloning, and part of this project's engineering work is extending that integration to expose and drive additional generation-control parameters from a Voice Style Profile.
4. **Consistency:** once built, a profile is applied automatically to every call for that business — this is what makes it personalization rather than a per-call manual setting.

7.3 Evaluation of the TTS component

- **Objective:** measured adherence of synthesized speech to the target profile (rate, pause timing) against reference audio.
- **Subjective:** AB/preference listening tests comparing stock voice vs. cloned voice (no style control) vs. cloned voice + Voice Style Profile, on brand consistency, naturalness, and contextual tone

appropriateness.

- **Consistency check:** variance of delivered rate/tone across many calls for the same business, confirming the profile is applied consistently rather than drifting per call.

8. Differentiation from a Normal Chatbot or RAG System

A standard RAG chatbot retrieves facts and answers questions; it does not learn how an organization characteristically communicates, and has no concept of vocal delivery beyond picking a voice. This project's difference:

- The LLM stage learns from the business's *own conversation history*, not just its documents — behavioral/stylistic patterns a retrieval system structurally cannot capture.
- Retrieval is kept, not replaced — a hybrid architecture with a stated division of labor, not a fine-tuning pitch.
- TTS is treated as a personalization surface in its own right, with a structured, reusable profile driving delivery — most voice-agent products (the base platform included, today) stop at "pick a voice."
- Both pipelines share a data source (`call_events`) and a style-feature extraction step, coupling "what to say" and "how to say it" instead of treating them as unrelated features.

9. Research Novelty vs. Engineering

Not novel (established technology, used as-is): LoRA/QLoRA fine-tuning, retrieval-augmented generation, neural TTS with voice cloning, function-calling LLM agents.

The actual contribution:

1. A concrete, evaluated data pipeline converting a voice-agent platform's own call logs into fine-tuning data and a communication style profile, with an empirical answer to *when personalization is worth it* rather than an assumption that it is.
2. A hybrid RAG+fine-tuning architecture with a stated, tested division of labor (facts vs. behavior), evaluated head-to-head at data volumes realistic for a small business.
3. The **Voice Style Profile** as a structured intermediate representation between what an LLM decided to say and how a TTS engine renders it — coupling delivery to per-turn conversational context and a business-specific profile extracted from data.

The value of the project is the engineering-plus-evaluation of a coherent pipeline that does not yet exist here, and a defensible answer about which approach is worth its complexity — not the invention of a new model or algorithm.

10. Research Questions

ID	Question
RQ1	Does fine-tuning an open-weight LLM on a business's historical call transcripts measurably improve style/tone fidelity and intent-handling consistency compared to prompt-only and RAG-only baselines, at data volumes realistic for an SMB?
RQ2	Does combining fine-tuning (style/behavior) with retrieval (facts) reduce hallucination on business-specific facts compared to fine-tuning alone, without losing the style gains of RQ1?
RQ3	Can a per-business Voice Style Profile, extracted from limited reference audio/data and applied automatically at synthesis time, produce speech perceived as more brand-consistent and contextually appropriate than a fixed stock voice, without a measurable latency regression?
RQ4	What is the minimum conversation-data volume at which fine-tuning-based personalization starts to outperform RAG-only prompting on style fidelity — the practical break-even point for a real deployment recommendation?

11. Evaluation Methodology

Component	Method	Baselines compared	Key metrics
LLM personalization	Held-out conversation eval + human/LLM-judge pairwise preference	Prompt-only, RAG-only, fine-tune-only, hybrid	Style-fidelity distance, tool-call accuracy, hallucination rate, judge win-rate
TTS personalization	Objective acoustic analysis + subjective listening test	Stock voice, cloned voice (no style), cloned + Voice Style Profile	Rate/pause adherence, MOS/AB preference, tone-appropriateness
End-to-end	Simulated calls through the actual base-platform pipeline	Current system vs. personalized configuration	Perceived quality, scripted-scenario task success, latency overhead
Data-volume sensitivity	Ablation over training-set size (real + synthetic)	—	Style-fidelity curve vs. data volume (RQ4)

All comparisons run against the same test-agent scenarios (representative business personas — e.g., a clinic receptionist and a retail support line — each in English and Arabic) to keep results comparable across conditions.

12. Expected Results

Framed as testable hypotheses, not guaranteed outcomes:

- The hybrid condition is expected to achieve the best combined score (style fidelity + factual accuracy), avoiding each standalone approach's specific weakness.
- Fine-tuning-only is expected to show measurably higher hallucination on facts that changed after training, demonstrating why retrieval remains necessary.

- RAG-only is expected to underperform on style-fidelity and tool-use-convention metrics, since retrieved text cannot change the model's own phrasing habits.
- The Voice Style Profile condition is expected to be preferred over a stock voice for brand-consistency judgments — with the size of that preference an open empirical question.
- A data-volume threshold below which fine-tuning gains disappear into noise is expected to exist and is directly actionable for deployment decisions.

13. FYP Scope and Feasibility

In scope

- Offline data pipeline: transcript export, cleaning, PII redaction, format conversion, style-feature extraction against the existing schema.
- QLoRA fine-tuning pipeline and evaluation harness on a small (7–8B) open model — not the production Groq model.
- A local inference server for the fine-tuned condition, wired into a modified copy of the worker's LLM stage for evaluation.
- Voice Style Profile extraction and a delivery-controller step feeding the existing Chatterbox integration, extended to pass through generation-control parameters.
- The evaluation suite of §11, run on 2–3 synthetic/pilot business personas × 2 languages.

Out of scope

- Fixing the base platform's unrelated issues (auth, observability, webhook durability).
- Production multi-tenant adapter-serving infrastructure at scale — discussed as future work only.
- Full fine-tuning, or fine-tuning of the 32B production model.
- Sourcing real production data from an operating business — pilot/synthetic data used instead, limitation stated openly.

Feasibility: every component maps to a known, bounded technique (QLoRA, PEFT, standard prosodic feature extraction, an existing self-hosted TTS server) rather than open-ended research, and reuses the base platform's existing config/provider-abstraction seams rather than new platform architecture. Risk sits in data preparation and evaluation design, not in the underlying methods.

14. Implementation Roadmap

Phase	Weeks	Deliverable
1	1–4	Data pipeline: transcript export/cleaning/PII-redaction/format-conversion scripts; style-feature extractor
2	5–7	Baselines: prompt-only and RAG-only evaluation harness; fact QA eval set built
3	8–12	QLoRA fine-tuning pipeline; per-persona adapters trained on pilot/synthetic data
4	13–15	Hybrid integration: fine-tuned model + retrieval tool call wired end-to-end
5	13–17 (parallel)	Voice Style Profile: prosodic extraction, delivery controller, extended Chatterbox integration
6	18–21	Full evaluation suite across all conditions/personas; data-volume ablation (RQ4)
7	22–24	Results analysis, thesis/report, demo

15. Risks and Mitigations

Risk	Mitigation
Insufficient real conversation data to fine-tune meaningfully	Validated synthetic augmentation; report the data-volume ablation (RQ4) as a first-class result
GPU compute constraints for fine-tuning	Small (7–8B) base model + QLoRA (4-bit), scoped to single-GPU/cloud-credit-scale compute from the outset
Style/emotion evaluation is inherently subjective	Combine objective acoustic metrics with structured, rubric-based listening tests
Chatterbox server's native parameter surface less flexible than expected	Degrade gracefully to rate/voice-selection only if emotion control proves unavailable; document the gap
Scope creep into fixing unrelated platform issues	§13's out-of-scope list is a hard boundary for the project

16. Conclusion

The base platform already has the two things a personalization project needs most: a real, working voice pipeline with clean seams for swapping the LLM and TTS stages, and a growing store of exactly the conversational data such a project would learn from. What it does not yet have is any mechanism to turn that data into a business-specific model or voice. This proposal is scoped to build and, more importantly, **evaluate** that mechanism — a hybrid fine-tuning-plus-retrieval approach for what the agent says, and a Voice Style Profile for how it says it — with an explicit, testable claim about when each technique is worth its cost, rather than treating either as self-evidently better.

References (indicative)

- Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," 2021.
- Dettmers et al., "QLoRA: Efficient Finetuning of Quantized LLMs," 2023.
- Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," 2020.
- Resemble AI, "Chatterbox," open-source TTS model, 2025.
- Qwen Team, "Qwen2.5/Qwen3 Technical Report."
- Base project documentation: README.md, docs/RESEARCH.md, docs/PLAN.md, docs/LATENCY.md, docs/architecture.md, docs/project-analysis.md, engineering audit notes.